

Soluciones del curso

Material de referencia del alumno

STACK DEL CURSO

- Apache Kafka 4.2 con KRaft (sin ZooKeeper)
- Confluent Platform 8.2.0
- OpenJDK 21 (Temurin)
- Docker Compose

Lab 08 usa CP 7.9.0 / Kafka 3.7 puntualmente porque Control Center Legacy requiere esa versión.

Índice

Lab 01 — Radiografía del clúster	3
Lab 02 — Pub/Sub en acción	11
Lab 03 — Construye tu propio clúster KRaft	16
Lab 04 — Operando tópicos	21
Lab 05 — Resiliencia y retención	26
Lab 06 — Productores afilados	32
Lab 07 — Consumer groups bajo presión	37
Lab 08 — Monitoreo con Control Center	42
Lab 09 — Kafka Connect	49
Lab 10 — Schema Registry y ksqlDB	55
Lab 11 — Prometheus y Grafana	63
Lab 12 — Seguridad y capstone final	69

LAB 01

Radiografía del clúster

Lab 01 — Reporte resuelto (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Datos del alumno

Campo	Valor
Nombre	[Solución de referencia]
Fecha	[Fecha de ejecución]
Sección	N/A

Sección 1: Componentes del clúster

Componente	Imagen Docker	Puerto externo	Rol / Función
kafka-broker-1	confluentinc/cp-kafka:8.2.0	9092	Broker y controlador KRaft (almacena datos, participa en quorum)
kafka-broker-2	confluentinc/cp-kafka:8.2.0	9093	Broker y controlador KRaft (almacena datos, participa en quorum)
kafka-broker-3	confluentinc/cp-kafka:8.2.0	9094	Broker y controlador KRaft (almacena datos, participa en quorum)
kafbat-ui	ghcr.io/kafbat/kafka-ui	8090	Interfaz web open-source para visualizar el clúster (brokers, tópicos, mensajes)

Componente	Imagen Docker	Puerto externo	Rol / Función
gps-producer	confluentinc/cp-kafka:8.2.0	N/A	Productor que genera datos GPS simulados de la flota NovaTech

Controlador KRaft

Dato	Valor
Leader ID	Varía (1, 2 o 3) - El primer broker que completa la elección
Votantes	1, 2, 3 (los tres brokers participan en el quorum)
Epoch actual	Depende de cuántas elecciones han ocurrido (generalmente 1 en un inicio limpio)

Sección 2: Distribución de particiones

Los líderes y réplicas varían según la ejecución. Ejemplo típico:

Partición	Broker líder	Réplicas	ISR	¿Todas sincronizadas?
0	1	1,2,3	1,2,3	Sí
1	2	2,3,1	2,3,1	Sí
2	3	3,1,2	3,1,2	Sí
3	1	1,3,2	1,3,2	Sí
4	2	2,1,3	2,1,3	Sí
5	3	3,2,1	3,2,1	Sí

Análisis de distribución

- ¿Los líderes están balanceados? **Sí, Kafka distribuye los líderes equitativamente entre los brokers disponibles**
- Particiones por broker (ejemplo):
 - Broker 1: 2 particiones líder
 - Broker 2: 2 particiones líder
 - Broker 3: 2 particiones líder

Sección 3: Tolerancia a fallos

Estado ANTES de la caída

(Mismos valores que la Sección 2 - todas las particiones con 3 réplicas en ISR)

Estado DESPUÉS de la caída (Broker 2 detenido)

Los valores son ilustrativos. Lo clave es que: - Las particiones que lideraba el Broker 2 ahora son lideradas por otro broker - El ISR de todas las particiones se reduce a 2 (los brokers que siguen activos)

Partición	Líder	ISR
0	1	1,3
1	3 (era 2)	3,1
2	3	3,1
3	1	1,3
4	1 (era 2)	1,3
5	3	3,1

Controlador activo: Si era el 2, cambia a 1 o 3. Si no era el 2, permanece igual.

Estado DESPUÉS de la recuperación

- Las ISR vuelven a incluir al Broker 2 (tres brokers en cada ISR)
- El Broker 2 **generalmente no recupera el liderazgo automáticamente** (depende de la configuración `auto.leader.rebalance.enable`)
- El quorum vuelve a tener 3 votantes activos

Observaciones

- ¿El productor GPS siguió funcionando? **Sí, el productor se reconectó a los brokers disponibles**
- ¿Se perdieron mensajes? **No, con `min.insync.replicas=2` y 2 brokers activos, el productor pudo seguir escribiendo**
- ¿El broker recuperado volvió a ser líder? **Generalmente no de inmediato, pero sí vuelve al ISR**
- Tiempo de resincronización: **Generalmente entre 10-30 segundos para el volumen de datos de este lab**

Sección 4: Conclusiones

¿Qué aprendí sobre la arquitectura de Kafka?

Kafka distribuye los datos de un tópico en múltiples particiones, y cada partición se replica en varios brokers. Este diseño permite tanto el paralelismo (múltiples particiones) como la durabilidad (múltiples réplicas). El modo KRaft elimina la dependencia de ZooKeeper, usando un quorum integrado para la gestión de metadatos.

¿Qué aprendí sobre la tolerancia a fallos?

Kafka puede tolerar la caída de un broker sin pérdida de datos ni interrupción del servicio, siempre que haya suficientes réplicas sincronizadas (ISR). La elección de nuevos líderes es automática y rápida. El broker recuperado se resincroniza automáticamente.

¿Qué rol juegan las réplicas ISR?

Las réplicas ISR (In-Sync Replicas) son copias de una partición que están completamente sincronizadas con el líder. Cuando el líder cae, solo una réplica ISR puede ser elegida como nuevo líder, garantizando que no se pierdan datos confirmados. El parámetro `min.insync.replicas` controla cuántas réplicas deben confirmar una escritura.

Sección 5: Desafío extra

Reto 1: Volumen de datos

Dato	Valor
Bytes totales	Varía según el tiempo de ejecución (~varios KB por minuto)
Comando	<code>docker exec kafka-broker-1 kafka-log-dirs --bootstrap-server kafka-broker-1:29092 --describe --topic-list novatech.fleet.gps</code>

Reto 2: Distribución de datos

Dato	Valor
Diferencia entre particiones	Debería ser relativamente uniforme

Dato	Valor
Hipótesis	El productor no especifica clave de partición, por lo que Kafka usa el particionador sticky por defecto, que distribuye de manera aproximadamente uniforme, aunque puede haber ligeras variaciones por el batching

Reto 3: Kafbat UI

Dato	Valor
Brokers visibles	3 (broker-1, broker-2, broker-3)
Controlador KRaft	Varía (1, 2 o 3, el primero electo)
Particiones del tópico GPS	6
Throughput	~0.5 mensajes/segundo (1 mensaje cada 2 segundos)
Consumer group <code>lab01-explorer</code>	Aparece si el alumno ya consumió mensajes

Solución de referencia - Lab 01

Lab 01 — Respuestas del desafío (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Reto 1: Volumen de datos

Comando

```
docker exec kafka-broker-1 kafka-log-dirs \
  --bootstrap-server kafka-broker-1:29092 \
  --describe \
  --topic-list novatech.fleet.gps
```

Explicación

El comando `kafka-log-dirs` muestra información detallada sobre los directorios de log de Kafka, incluyendo el tamaño en bytes de cada partición en cada broker. La salida es un JSON que contiene:

- `broker` : ID del broker consultado
- `logDirs` : Array con los directorios de log
- Para cada partición: `size` (bytes), `offsetLag`, `isFuture`

Para obtener el total de bytes, el alumno debe: 1. Sumar el campo `size` de todas las particiones del tópico `novatech.fleet.gps` 2. Tener en cuenta que cada partición tiene réplicas en otros brokers, por lo que el volumen total "lógico" es el tamaño de las particiones líderes, no la suma de todas las réplicas

Forma alternativa de calcular (más precisa)

```
# Ver tamaño por partición en cada broker
for BROKER in kafka-broker-1 kafka-broker-2 kafka-broker-3; do
  echo "=== $BROKER ==="
  docker exec $BROKER kafka-log-dirs \
    --bootstrap-server $BROKER:29092 \
    --describe \
    --topic-list novatech.fleet.gps
done
```


El volumen total depende del tiempo que lleve corriendo el productor. Con un mensaje cada 2 segundos y un tamaño promedio de ~150 bytes por mensaje JSON:

- 1 minuto: ~4.5 KB (30 mensajes)
 - 10 minutos: ~45 KB (300 mensajes)
 - 1 hora: ~270 KB (1800 mensajes)
-

Reto 2: Distribución de datos

Análisis

El productor GPS del laboratorio no especifica una clave de partición (`--key`), por lo que Kafka usa el **particionador sticky por defecto** (a partir de Kafka 2.4+/KIP-480).

Comportamiento del sticky partitioner: - Agrupa mensajes del mismo batch en la misma partición - Cuando el batch se completa o se envía, elige una nueva partición - Resultado: distribución aproximadamente uniforme a lo largo del tiempo, pero con posibles diferencias en ventanas cortas

¿Por qué podría haber diferencias?

1. **Particionador sticky:** Al agrupar mensajes por batch, las particiones con batches más recientes pueden tener ligeramente más datos
2. **Tiempo de inicio:** Si el productor acaba de arrancar, puede que no todas las particiones hayan recibido la misma cantidad de mensajes
3. **Round-robin puro vs. sticky:** El `kafka-console-producer` envía un mensaje a la vez (sin batching real), por lo que con la versión de Kafka usada, la distribución tiende a ser bastante uniforme

Conclusión esperada

La diferencia entre la partición con más datos y la de menos datos debería ser mínima (pocos mensajes de diferencia). Si un alumno ve diferencias grandes, puede indicar que el productor se reinició recientemente.

Reto 3: Exploración visual con Kafbat UI

Navegación paso a paso

1. Abrir `http://localhost:8090` en el navegador
2. La pantalla inicial muestra el clúster `novatech-cluster` pre-configurado vía variables de entorno (no requiere configuración manual)
3. Hacer clic en el clúster para entrar al panel principal

Ubicación de los datos solicitados

- **Brokers:** Menú lateral > "Brokers"
- Lista los 3 brokers con su ID, host, puerto y estado
- Marca cuál es el "Controller" del quorum KRaft
- **Tópicos:** Menú lateral > "Topics"
- `novatech.fleet.gps` aparece con sus 6 particiones
- Al hacer clic, se accede a sub-secciones: Overview, Messages, Consumers, Settings
- **Throughput del tópico:**
- Topics > `novatech.fleet.gps` > pestaña "Overview"
- El gráfico "Messages per second" muestra ~0.5 msg/seg
- **Mensajes en vivo:**
- Topics > `novatech.fleet.gps` > pestaña "Messages"
- Permite filtrar por partición, offset o por contenido (CEL filtering)
- Cada mensaje JSON GPS aparece formateado y legible
- **Consumer Groups:**
- Menú lateral > "Consumers"
- El grupo `lab01-explorer` aparece si el alumno ya ejecutó `consume-gps.sh`
- Muestra lag por partición, miembros y offset actual

Respuestas esperadas

Dato	Valor esperado
Brokers visibles	3 (broker-1, broker-2, broker-3)
Controlador KRaft	Varía (1, 2 o 3, el primero electo)
Particiones del tópico GPS	6
Throughput	~0.5 msg/seg (1 mensaje cada 2 segundos)
Consumer group <code>lab01-explorer</code>	Aparece si el alumno consumió mensajes; si no, la sección está vacía

LAB 02

Pub/Sub en acción

Lab 02 — Reporte resuelto (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Parte 1: El log inmutable

Pregunta	Respuesta esperada
¿Cuántos mensajes leíste la primera vez?	5 (los 5 que produjo manualmente)
¿Aparecieron de nuevo?	Sí , los mismos 5 mensajes
Sin <code>--from-beginning</code> , ¿qué mensajes ves?	Solo los nuevos producidos mientras el consumer está activo
¿Por qué Kafka se comporta así?	Kafka es un log inmutable , no una cola. Los mensajes se mantienen hasta que la política de retención los elimine (por tiempo o tamaño). Múltiples consumidores pueden leerlos múltiples veces

Parte 2: Pub/Sub con múltiples consumidores

Pregunta	Respuesta esperada
¿Cuántas terminales recibieron el mensaje?	Las 3
¿En qué orden llegaron?	Casi simultáneo en las 3, mismo orden

Pregunta	Respuesta esperada
¿Qué habría pasado en RabbitMQ?	Solo UNA habría recibido el mensaje (modelo cola con consumo competitivo)
¿Apareció algún grupo?	No, porque consumir SIN grupo usa grupos efímeros que no se persisten

Parte 3: Consumer Groups

Distribución esperada

Cantidad de consumidores	Particiones por consumidor	Total
1	6	6
2	3 y 3	6
3	2, 2, 2	6
5	2, 1, 1, 1, 1	6 (uno tiene 2 porque 6/5 no es entero)

Respuestas

Pregunta	Respuesta esperada
¿Algún mensaje fue recibido por más de un consumidor del mismo grupo?	No . Dentro del grupo, cada partición pertenece a UN solo miembro
Con 5 consumidores y 6 particiones, ¿hay alguno ocioso?	No (los 5 tienen al menos 1 partición)
¿Qué pasaría con 7 consumidores?	Un miembro queda ocioso (sin particiones asignadas)
¿Se redistribuyeron las particiones al cerrar uno bruscamente?	Sí , después del rebalanceo (~5-10 segundos)

Parte 4: Offsets y replay

Pregunta	Respuesta esperada
¿El grupo <code>reportes</code> empezó desde el inicio o el final?	Por defecto desde el FINAL (<code>auto.offset.reset=latest</code>). Solo recibe mensajes producidos DESPUÉS de su creación

Pregunta	Respuesta esperada
¿Qué CURRENT-OFFSET tienen las particiones después del reset?	0 en todas
¿El reset de <code>reportes</code> afectó al grupo <code>alertas</code> ?	No. Cada grupo tiene sus propios offsets, son independientes

Parte 5: Desafío - Claves y particionado

Predicción vs realidad

Las particiones exactas dependen del hash, pero lo importante es la **consistencia**: - La misma clave siempre va a la misma partición. - La predicción del helper (cksum) puede no coincidir con la real (murmur2). Es una diferencia esperada.

Reflexión

Pregunta	Respuesta esperada
¿Los 4 eventos de NVT-1001 cayeron en la misma partición?	Sí, siempre. Esa es la garantía de Kafka
Con 100 vehículos y 6 particiones, ¿cuántos comparten partición?	~17 vehículos por partición en promedio (100/6)
¿Eso rompe el orden por vehículo?	No. Dentro de una partición todo está ordenado. Los vehículos que comparten partición coexisten en orden cronológico, pero el orden POR vehículo se mantiene

Solución - Lab 02

Lab 02 — Respuestas del desafío (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearte con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Reto 1-3: Particionado por clave

Comportamiento esperado

1. **Predicción con cksum vs realidad murmur2:** las particiones específicas pueden diferir entre la predicción del script y lo que aparece en Kafbat UI. Esto es normal: `cksum` es una aproximación didáctica.
2. **Consistencia interna:** lo crítico es que CADA clave siempre vaya a la MISMA partición: - Si NVT-1001 cayó en la partición 3, los 4 eventos de NVT-1001 estarán en la partición 3. - Si NVT-1002 cayó en la partición 5, todos sus eventos estarán en la 5.
3. **Verificación visual en Kafbat UI:** - **Topics** > **novatech.fleet.events** > **Messages** - Filtrar mensajes por columna `Key` - Confirmar que `Partition` es siempre el mismo para una clave dada

Reto 4: Reflexión sobre 100 vehículos en 6 particiones

Distribución

Con 100 claves distribuidas uniformemente en 6 particiones (asumiendo buen hash): - Promedio: ~17 vehículos por partición - Cada vehículo comparte su partición con ~16 otros vehículos

Implicaciones del orden

El orden por vehículo se mantiene: - Dentro de una partición, los mensajes están en orden cronológico de llegada - Si NVT-1001 envía sus eventos en orden A, B, C, todos llegan a la misma partición y en ese orden - Aunque NVT-1042 también esté en esa partición, sus eventos no rompen el orden de NVT-1001 (Kafka mantiene el orden de inserción)

El orden global NO se mantiene: - Si NVT-1001 emite un evento A en partición 3, y NVT-1002 emite un evento B en partición 5, no podemos garantizar que A se procese antes que B desde la perspectiva del consumidor multi-partición

Cuándo importa cuántas particiones

Para garantizar **una partición por entidad** (raro y normalmente innecesario): - Necesitarías al menos N particiones para N entidades, asumiendo distribución perfecta del hash - En la práctica, debido a colisiones de hash, se recomienda 2-3x el número de entidades únicas - **Esto casi nunca se hace.** Las colisiones de hash son aceptables porque el orden se garantiza POR clave, no por entidad única en partición única

Soluciones del desafío - Lab 02

LAB 03

Construye tu propio clúster KRaft

Lab 03 — Reporte resuelto (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Parte 1: Anatomía de la imagen Confluent

Pregunta	Respuesta esperada
Binarios <code>kafka-*</code> en <code>/usr/bin/</code>	Aproximadamente 30+. Los más relevantes: <code>kafka-topics</code> , <code>kafka-console-producer</code> , <code>kafka-console-consumer</code> , <code>kafka-consumer-groups</code> , <code>kafka-storage</code> , <code>kafka-metadata-quorum</code> , <code>kafka-broker-api-versions</code> , <code>kafka-configs</code> , <code>kafka-acls</code> , etc.
Directorio de configs de ejemplo	<code>/etc/kafka/</code> con <code>server.properties</code> , <code>broker.properties</code> , <code>controller.properties</code> (en CP 8.2 ya no existe el subdirectorio <code>kraft/</code>). En runtime el endpoint genera <code>/etc/kafka/kafka.properties</code> con la config efectiva.
Contenido aproximado de <code>server.properties</code>	Configuración tipo <code>process.roles=broker,controller</code> , <code>node.id=1</code> , listeners por defecto, <code>log.dirs=/tmp/kraft-combined-logs</code> (será reemplazado por env vars al levantar)
Versión de Java	OpenJDK 21 (Temurin 21.0.10 en CP 8.2.0)

Parte 2: Mi primer broker solitario

Pregunta	Respuesta esperada
CLUSTER_ID	Valor único generado por <code>kafka-storage random-uuid</code> . Ejemplo: <code>MKU30EVBNTcwNTJENDM2Qk</code>

Pregunta	Respuesta esperada
Broker sin formatear	Si la imagen Confluent no detecta storage formateado, intenta autoformatear si pasaste CLUSTER_ID por env. Si falla, debes ejecutar <code>kafka-storage format</code> manualmente
Qué hace <code>kafka-storage format</code>	Crea el archivo <code>meta.properties</code> en el <code>log.dirs</code> , registrando el <code>cluster.id</code> y <code>node.id</code> . Es como "particionar" el disco para Kafka
¿Por qué RF=1 con 1 broker?	Replication factor mayor al número de brokers vivos genera el error <code>InvalidReplicationFactorException</code> porque no hay suficientes brokers para colocar las réplicas
Comando para verificar broker vivo	<code>kafka-broker-api-versions --bootstrap-server kafka-broker:29092</code>

Parte 3: Creciendo a 3 brokers

QUORUM_VOTERS

```
1@kafka-broker-1:39092,2@kafka-broker-2:39093,3@kafka-broker-3:39094
```

Respuestas

Pregunta	Respuesta esperada
¿Por qué los 3 deben compartir CLUSTER_ID?	Es la identidad del clúster. Si difieren, KRaft los considera clústers distintos y no se reconocen
¿Por qué puertos CONTROLLER distintos?	Cada broker corre en su propio contenedor, pero comparten la red Docker. Aunque los puertos sean internos, no pueden colisionar dentro del namespace de la red
Active Controller	Varía: cualquiera de los 3 (1, 2 o 3). El primero en arrancar y completar la elección
Criterio de elección	KRaft usa el algoritmo Raft. El primero que alcanza el quorum y publica un voto vence. En la práctica suele ser el de menor <code>node.id</code> que arranca a tiempo

Parte 4: Chequeo de salud KRaft

Pregunta	Respuesta esperada
Voters	3
LeaderId	1, 2 o 3 (varía)
Lag	Idealmente 0 o muy bajo
<code>--replication</code>	Muestra el estado de replicación del log de metadatos por cada réplica del quorum

Parte 5: Desafío - Listeners separados

Pregunta	Respuesta esperada
Listeners en mismo puerto	El broker se niega a arrancar: <code>IllegalArgumentException: requirement failed: Each listener must have a different port</code> . Esto previene conflictos de socket
EXTERNAL anunciado como <code>kafka-broker-1</code>	El cliente del host no puede resolver <code>kafka-broker-1</code> porque ese hostname solo existe dentro de la red Docker. Por eso <code>localhost</code> es la dirección correcta para clientes externos
INTER_BROKER vs CONTROLLER	El tráfico de datos (entre brokers, replicación) y el tráfico de control (quorum, metadatos) se aíslan. Permite aplicar políticas distintas: cifrado, autenticación, QoS

Solución - Lab 03

Lab 03 — Respuestas del desafío (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearte con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Reto 1: Listeners en el mismo puerto

Error esperado en logs:

```
java.lang.IllegalArgumentException: requirement failed: Each listener must have a different port
```

Por qué: cada listener mantiene un socket TCP propio. Dos listeners no pueden bindear el mismo puerto al mismo tiempo.

Implicación pedagógica: en producción, esto previene que una misconfiguración accidental rompa el aislamiento entre tráfico de datos y de control.

Reto 2: Advertised listener mal configurado

Síntoma: el cliente desde el host no logra completar el handshake con el broker. Después de la conexión inicial, el broker le responde "para producir/consumir, conéctate a `kafka-broker-1:9092`", pero ese hostname no es resoluble desde el host.

Solución correcta: el `advertised.listener` debe ser **lo que el cliente puede resolver**: - Si el cliente está dentro de la red Docker → hostname interno (`kafka-broker-1`) - Si el cliente está en el host → `localhost` (o la IP pública si es remoto)

Por eso usamos dos listeners distintos: - `PLAINTEXT` con advertised `kafka-broker-1:29092` → para clientes dentro de Docker - `EXTERNAL` con advertised `localhost:9092` → para clientes en el host

Reto 3: Reflexión

¿Por qué CONTROLLER separado? - KRaft replica el log de metadatos del clúster (parecido a etcd o Raft puro). Este tráfico es de bajo volumen pero crítico. - Aislarlo en su propio listener permite aplicar políticas de seguridad y QoS distintas a las del tráfico de datos del cliente. - Además, en arquitecturas dedicadas (controllers separados de brokers), esos nodos solo abren el listener CONTROLLER, no el PLAINTEXT.

INTER_BROKER vs CONTROLLER - Entre brokers, viaja la replicación de tópicos (datos). Es el grueso del tráfico. - Entre controllers, viaja el log de metadatos (raft). Es bajo volumen pero crítico. - Separarlos permite, por ejemplo, usar TLS/SASL solo en CONTROLLER si quieres asegurar el plano de control sin pagar el costo de cifrado en el plano de datos.

Listener apropiado por origen del cliente - Cliente en otro broker dentro de Docker →

`INTER_BROKER` (PLAINTEXT) - Cliente en el quorum (controller) → `CONTROLLER` - Productor o consumidor desde el host → `EXTERNAL`

Soluciones del desafío - Lab 03

LAB 04

Operando tópicos

Lab 04 — Reporte resuelto (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Parte 1: Anatomía de un tópico

Pregunta	Respuesta esperada
Tópicos sin <code>--internal</code>	1 (<code>novatech.fleet.gps</code>)
Tópicos con <code>--internal</code>	2 o 3 (incluye <code>__consumer_offsets</code>)
¿Para qué sirve <code>__consumer_offsets</code> ?	Tópico interno donde Kafka guarda los offsets actuales de cada consumer group, partición por partición

Configuraciones de ejemplo (varían según el broker)

Las configs típicas incluyen: `retention.ms` (DEFAULT_CONFIG = 604800000), `cleanup.policy` (DEFAULT = delete), `compression.type` (DEFAULT = producer), `segment.ms`, `min.insync.replicas`, etc.

Parte 2: Tópicos con personalidad

Tópico	Particiones	retention.ms	compression	min.insync
<code>novatech.gps.realtime</code>	12	3600000 (1h)	lz4	(default)
<code>novatech.audit.events</code>	6	7776000000 (90d)	gzip	2
<code>novatech.vehicle.state</code>	6	(n/a, compact)	(default)	(default)
<code>novatech.alerts.critical</code>	3	-1 (∞)	(default)	3

Razonamientos clave

- **12 particiones GPS**: alto volumen, alto paralelismo de consumo
- **gzip en audit**: datos accedidos poco frecuentemente, prioriza compresión
- **compact en vehicle.state**: solo importa el último estado por clave
- `min.insync.replicas=3` con **RF=3**: si UN broker se cae, no se puede escribir (la app debe manejar `NotEnoughReplicasException`)
- `unclean.leader.election.enable=false`: previene que un follower fuera-de-ISR sea elegido líder, evitando pérdida de datos

Compactación

Después de 5 mensajes con clave NVT-1001, el alumno **probablemente verá los 5** porque la compactación es asíncrona y no se dispara inmediatamente. En producción, se controla con `min.cleanable.dirty.ratio`, `segment.ms` y el tiempo de espera del log cleaner. Lo correcto es explicar que la compactación garantiza que **eventualmente** quedará solo el último mensaje por clave.

Parte 3: Modificar tópicos en caliente

Pregunta	Respuesta esperada
¿ <code>retention.ms</code> cambió?	Sí, a 31536000000 (365 días)
¿ <code>ConfigSource</code> ?	Cambió a <code>DYNAMIC_TOPIC_CONFIG</code>
¿Sin downtime?	Sí, todo en caliente
¿Particiones tras aumento?	18
¿Particiones 12-17 tienen mensajes?	No (todavía). Solo las nuevas escrituras llegarán a ellas según el hash
Error al disminuir	"The number of partitions for a topic can only be increased"
¿Por qué?	Disminuir rompería el orden por clave (mensajes existentes en particiones 12-17 no se pueden mover sin perder consistencia)
¿ <code>retention.ms</code> tras <code>--delete</code> ?	604800000 (7 días, el default del broker)

Parte 4: Producción y consumo masivo

Métricas típicas (Mac M1/M2, Docker)

- 5.000 mensajes en 5-15 segundos → ~300-1000 msg/seg

- `perf-test` con `acks=all` : 5.000-20.000 msg/seg
- `perf-test` con `acks=1` : 10.000-40.000 msg/seg
- Diferencia: `acks=1` es típicamente 1.5-3x más rápido

Consumo desde el principio

- Los mensajes NO vienen ordenados globalmente. Vienen ordenados POR PARTICIÓN.
- El alumno verá saltos entre claves (por ejemplo: NVT-3, NVT-3, NVT-7, NVT-7, NVT-3...) porque el consumer va leyendo round-robin entre particiones.

Consumo de partición específica

- En la partición 3, el alumno verá un subconjunto consistente de claves (mismas o pocas distintas).
- La razón es que `hash(key) % 12` envía cada clave SIEMPRE a la misma partición.

`acks=all` vs `acks=1`

- `acks=1` : el productor espera solo el ACK del líder. Más rápido pero si el líder muere antes de replicar, se pierde el mensaje.
- `acks=all` : espera ACK de todas las réplicas en ISR. Más lento pero durable.

Parte 5: Desafío

Pregunta	Respuesta esperada
¿Tópico tras eliminar?	No aparece (eliminación efectiva en segundos)
¿RF subió de 1 a 3?	Sí, después del plan de reasignación
¿Por qué no con <code>--alter</code> ?	Cambiar RF mueve datos físicos entre brokers; debe ser controlado para no saturar la red
¿Más peligroso: particiones o RF?	RF es más peligroso operacionalmente porque genera tráfico masivo de replicación
Política para pagos	<code>min.insync.replicas=2</code> con RF=3 es lo estándar. Permite tolerar 1 falla y aún escribir. <code>min.insync.replicas=3</code> es excesivo (no tolera ninguna falla)

Lab 04 — Respuestas del desafío (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo. Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Reto 1: Eliminar tópico

- Eliminación lógica: inmediata (el tópico desaparece de `--list`)
- Eliminación física: depende de `delete.topic.enable=true` (default en Kafka 4.x) y del cleanup interno (segundos a minutos)
- Si el tópico tenía consumer groups activos, esos grupos quedan con offsets huérfanos

Reto 2: Aumentar RF con plan de reasignación

El plan JSON especifica explícitamente qué brokers tendrán cada partición. Ejemplo de salida tras `--verify`:

```
Status of partition reassignment:
Reassignment of partition novatech.test.rf1-0 is completed.
Reassignment of partition novatech.test.rf1-1 is completed.
Reassignment of partition novatech.test.rf1-2 is completed.
```

Después del cambio, `describe-topic.sh` muestra:

```
Topic: novatech.test.rf1
PartitionCount: 3
ReplicationFactor: 3
Topic: novatech.test.rf1 Partition: 0 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
...
```

¿Por qué requiere plan explícito?

- Cambiar RF implica COPIAR datos físicos entre brokers
- El admin debe controlar:
- **Cuándo** ocurre (ventana de mantenimiento)
- **A qué brokers** se mueve (no saturar uno)
- **Throttling:** `--throttle <bytes/sec>` para no afectar producción
- `kafka-topics --alter` no tiene esta capacidad de control

Reto 3: Reflexión

¿Más peligroso: particiones o RF?

RF es más peligroso operacionalmente: - Aumentar particiones: cambia hash routing pero no mueve datos antiguos. Es CPU/coordinación. - Aumentar RF: copia físicamente todos los datos del tópico a más brokers. Genera tráfico masivo de replicación. Puede saturar la red interna y afectar la latencia de producción.

Rollback de retention

- Aplicar `--alter` con el valor anterior. **Pero:** los mensajes que ya se eliminaron por la retention nueva NO vuelven.
- Por eso se recomienda: ANTES de bajar retention, exportar/respaldar lo crítico.

`min.insync.replicas` para pagos

Recomendación: `min.insync.replicas=2` con **RF=3**

Razones: - Tolerancia a 1 falla simultánea (broker caído por mantenimiento) - Si exiges 3, cualquier mantenimiento bloquea las escrituras - 2/3 es el balance estándar entre durabilidad y disponibilidad - Las apps deben manejar `NotEnoughReplicasException` con retry

Soluciones del desafío - Lab 04

LAB 05

Resiliencia y retención

Lab 05 — Reporte resuelto (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Parte 1: ISR bajo el microscopio

Estado inicial esperado

Las 6 particiones tienen `Replicas: [N1, N2, N3]` y `ISR: [N1, N2, N3]` (mismo conjunto, en algún orden). Los líderes están distribuidos: típicamente 2 por broker.

Pregunta	Respuesta esperada
ISR == Replicas	Sí (clúster sano)
MIR del tópico	2
Productor siguió tras tumbar broker 2	Sí (gracias a MIR=2 y 2 brokers vivos)
ISR cambió	Sí: las réplicas en broker 2 salieron del ISR
Particiones que cambiaron líder	Las que tenían a broker 2 como líder (típicamente 2 de 6)
Al revivir, broker 2 volvió al ISR	Sí, después del catch-up
¿Recuperó liderazgo?	NO automáticamente. Kafka mantiene el líder actual. Para forzar rebalanceo: <code>kafka-leader-election --election-type PREFERRED</code>

Parte 2: MIR vs ISR

Pregunta	Respuesta esperada
Estricto con 3 brokers: produjo bien	Sí
ISR del estricto tras kill	2
2 < 3: ¿bloquea?	Sí
Error en estricto	<code>NotEnoughReplicasException</code> o timeout
¿Por qué bloquea?	Kafka prioriza durabilidad sobre disponibilidad cuando MIR no se cumple
Resiliente con 2 brokers vivos	Funciona porque MIR=2
Estricto al revivir	Vuelve a aceptar escrituras

Reflexión

- **MIR=3 con RF=3 nunca tolera ninguna falla:** cualquier mantenimiento bloquea las escrituras. Demasiado estricto.
- **MIR=2 con RF=3** es el balance estándar: tolera 1 falla, mantiene durabilidad de 2/3.
- **Sistema de pagos:** típicamente RF=3, MIR=2, `acks=all`. La app debe manejar `NotEnoughReplicasException` con retry exponencial.

Parte 3: Recuperación

Pregunta	Respuesta esperada
5K mensajes con broker caído	Producción ~5-15s, sin problemas
Catch-up al revivir	5-30 segundos según volumen
ISR simultáneo	Generalmente sí, pero puede ser de a una en clúster grande
Recuperó liderazgo	NO. El nuevo líder se mantiene
Total mensajes	Suma de offsets de las 6 particiones (~5000+)
Coincide	Sí
¿Por qué no se perdieron?	<code>acks=all</code> + MIR=2: las escrituras se confirmaron solo cuando 2 réplicas las recibieron. Al volver, broker 2 las copió del líder

Parte 4: Retención

Comportamiento esperado

- **Después de producir y esperar 90s sin más actividad:** probablemente NO hay eliminación. El segmento activo no se cerró porque no hubo nuevos mensajes que fueren segment.ms.
- **Con producción periódica:** los segments se cierran y se eliminan los más viejos. El offset más antiguo (`--time -2`) sube de 0 a algún número positivo.
- **Diferencia de tamaño:** `efimero` ocupa muy poco (segmentos viejos eliminados). `resiliente` mantiene todo (retención 7 días default).

Conceptos clave

- Kafka no elimina mensajes individuales; elimina **segmentos completos**.
- `segment.ms` define cuándo se cierra un segmento por tiempo.
- `segment.bytes` lo cierra por tamaño (default 1 GB).
- Solo los segmentos CERRADOS pueden ser eliminados por retention.

Parte 5: Compactación y tombstones

Comportamiento esperado

Aspecto	Esperado
Mensajes producidos en estado	30 (6 rounds × 5 vehículos)
Mensajes leídos tras compactación	Variable: típicamente 5-15 (depende de cuándo se compactó)
Claves distintas	5 (NVT-1 a NVT-5)
Vehículo aparece varias veces	Posible: la compactación es asíncrona, no instantánea

Tombstone para NVT-3

- Inmediatamente después: NVT-3 con valores anteriores SIGUE apareciendo + el tombstone (NULL_VALUE)
- Después de 60s + nueva compactación: NVT-3 desaparece de la salida (incluyendo el tombstone si pasó `delete.retention.ms`)
- En este lab `delete.retention.ms` es default (24h), así que el tombstone seguirá visible un rato

Reflexión

Pregunta	Respuesta esperada
Casos ideales	Estado actual de entidades (perfil de usuario, configuración, último precio, etc.)
¿Por qué KEY obligatorio?	Sin clave no se puede determinar "el último valor" para compactar
Mensaje sin clave en compactado	Permanece indefinidamente (la compactación lo ignora)
Tombstones con retention propio	Para que consumers lentos puedan ver el "evento de eliminación" antes de que desaparezca

Solución - Lab 05

Lab 05 — Respuestas del desafío (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Reto 1-2: Compactación

La compactación es asíncrona. Aún con `min.cleanable.dirty.ratio=0.01`, depende de: - Que el log cleaner thread se active - Que haya al menos 1 segmento "viejo" (no el activo) - Que el ratio de dirty/total supere el umbral

En el lab, después de 30 mensajes y unos 30s, lo más probable es que el alumno vea **menos de 30 pero más de 5**, porque algunas claves duplicadas todavía no se compactaron.

Reto 3: Tombstones

Mecánica detallada

1. **Producir tombstone:** Kafka acepta un mensaje con `value=null` para una clave existente.
2. **Antes de compactación:** el tombstone aparece como un mensaje más en el log (con valor null).
3. **Durante compactación:** Kafka identifica que la clave tiene un tombstone como último valor → marca para eliminación.
4. **Resultado:** - Todos los mensajes anteriores de esa clave se borran. - El tombstone permanece visible durante `delete.retention.ms` (default 24h). - Después, también se borra.

Verificación visual

En Kafbat UI > Topics > `novatech.lab05.estado` > Messages: - Antes del tombstone: ves múltiples mensajes con clave NVT-3. - Después del tombstone (sin compactación): ves los anteriores + el tombstone (valor null). - Después de compactación: solo ves el tombstone (los anteriores desaparecieron). - Después de 24h: ya no ves nada de NVT-3.

Reto 4: Reflexión

Casos de uso ideales para compactación

- **Estado de entidades:** último estado conocido por ID
- **Configuraciones por aplicación:** última versión de la config
- **Cache materializado:** para reconstruir cache desde Kafka al iniciar una app

- **Cambio de propiedad:** último dueño de un recurso

Casos NO ideales

- **Eventos:** cada evento es único, no quieres que se "compacte" sobre eventos previos
- **Logs de auditoría:** necesitas el historial completo, no solo el último

Por qué KEY es obligatorio

La compactación se basa en agrupar por clave. Sin clave, no hay agrupación posible. Kafka simplemente IGNORA mensajes sin clave en tópicos compactados (los deja indefinidamente).

Tombstones con retention propio

Imagina un consumer que estuvo offline 1 hora. Cuando vuelve, necesita saber que NVT-3 fue eliminado para limpiar su cache local. Si el tombstone se borrara inmediatamente, el consumer nunca se enteraría. `delete.retention.ms` (24h por default) le da una ventana.

Soluciones del desafío - Lab 05

LAB 06

Productores afilados

Lab 06 — Reporte resuelto (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Parte 1: Tuning de batch.size y linger.ms

Métricas típicas (Mac M1/M2 con Docker)

Configuración	records/sec	Latencia p99
Baseline (defaults)	5.000 - 15.000	50-200 ms
Batch 64KB + linger 10ms	20.000 - 60.000	30-100 ms
Compresión LZ4	10.000 - 25.000	50-150 ms
Combinación pro (todo activado)	50.000 - 100.000+	30-80 ms
Máximo extremo (1MB batch + zstd)	80.000 - 150.000+	100-500 ms

Reflexión

- **Mejor throughput:** combinación pro con batch grande + linger + compresión.
- **Peor latencia:** máximo extremo (1MB batch + 100ms linger). El batch demora más en cerrarse.
- **Trade-off:** linger.ms agrega latencia por diseño (espera para acumular). Más allá de cierto punto, no compensa.
- **NovaTech GPS:** throughput alto, latencia tolerable (datos en streaming, no en milisegundos críticos) → batch + linger 10-50ms + LZ4.
- **Pagos en tiempo real:** latencia crítica, throughput menos. Batch chico, linger=0, sin compresión adicional.

Parte 2: Niveles de acks

Métricas típicas

Acks	Throughput	Latencia p99	Pérdida
0	60K-100K msg/s	5-20 ms	Sí (productor cae sin saber)
1	30K-60K msg/s	20-50 ms	Sí (líder muere antes de replicar)
all	15K-30K msg/s	50-150 ms	Solo si caen TODAS las réplicas en ISR

Reflexión

- **acks=0** es 3-5x más rápido que **acks=all**.
- **acks=0**: métricas, telemetría no crítica, logs de aplicación.
- **acks=1**: workloads donde la pérdida ocasional es aceptable (eventos de UI, clics).
- **acks=all**: pagos, auditoría, eventos críticos.

Parte 3: Idempotencia

Comportamiento esperado

- **Naive**: en local sin congestión, lo más probable es que el alumno NO vea duplicados con `timeout=100ms` y red local. Para forzarlos, bajar a 50ms o agregar latencia con `tc`.
- **Idempotente**: SIEMPRE produce exactamente N mensajes nuevos.

Configs forzadas

`enable.idempotence=true` requiere: - `acks=all` - `retries=Integer.MAX_VALUE` - `max.in.flight.requests.per.connection<=5`

Si forzas `acks=1` con idempotencia, el productor **falla al iniciar** con: `ConfigException: Must set acks to all in order to use the idempotent producer`.

Garantías

- **Idempotencia garantiza**: no-duplicados dentro de UN productor + UNA partición + UNA sesión.
- **NO garantiza**: no-duplicados a través de sesiones (productor reiniciado), ni a través de múltiples particiones.
- **Para exactly-once cross-partition**: necesitas transacciones.

Parte 4: Transacciones

Comportamiento esperado

- Con COMMIT: ambos `read_committed` y `read_uncommitted` ven los mensajes.
- Con ABORT (en la implementación pedagógica de este lab): el comportamiento puede no ser perfecto desde CLI; los alumnos verían diferencia entre los dos isolation levels si usaran código de aplicación con `producer.abortTransaction()`.

Limitación reconocida

`kafka-console-producer` no soporta control completo de transacciones. `kafka-verifiable-producer` tampoco maneja abort directamente. Esta parte del lab es **conceptual**: el alumno aprende QUÉ son las transacciones y cómo se inspeccionan, no necesariamente cómo programarlas.

Parte 5: Desafío

Particionado por clave fija

- Las 50K con la misma clave caen en UNA partición → un solo broker líder maneja todo el tráfico.
- Throughput cae significativamente vs distribución uniforme.

RoundRobinPartitioner

- Distribuye estrictamente. En general da throughput similar a sticky pero con peor latencia (más overhead de coordinación de batches por partición).

Hot partitioning

- 6 VIPs con 80% del tráfico → 6 particiones quedan saturadas, las demás ociosas.
- **Soluciones**: usar particionador custom que detecte la "calidez" de la clave y la distribuya, o cambiar el esquema de claves (concatenar VIP con timestamp para distribuir).

Cuándo no importa orden por clave

- Métricas agregadas (sumas, contadores) — el orden no afecta el resultado.
 - Eventos idempotentes con timestamp.
 - Cuando el procesamiento es por batch, no por evento.
-

Lab 06 — Respuestas del desafío (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearte con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Reto 1: Particionado por clave fija

Comportamiento

Cuando produces 50.000 mensajes con la MISMA clave (`K`), todos van a la misma partición porque `partition = hash(key) % num_partitions`. Verificable con:

```
kafka-cli/describe-topic.sh novatech.tuning.bench
```

Verás que solo una partición tiene `End Offset` alto; las demás siguen igual.

Throughput

Con clave fija: típicamente **30-50% del throughput** comparado con distribución uniforme. Razón: un solo broker líder hace todo el trabajo de esa partición; los demás están ociosos.

Reto 2: RoundRobinPartitioner vs Default

Default (sticky)

El particionador default desde Kafka 3.x (vigente en 4.x) usa **sticky batching**: agrupa mensajes en batches por partición, cambiando de partición solo cuando el batch se cierra. Esto da mejor throughput y mejor compactación de batches.

RoundRobinPartitioner

Distribuye estrictamente uno por uno entre particiones. Resultado: batches más pequeños, peor agrupación, **peor throughput** (a veces 20-40% menor).

Cuándo usar cada uno

- **Default (sticky):** 99% de los casos. Mejor throughput.

- **RoundRobinPartitioner**: cuando necesitas distribución estrictamente uniforme y no te importa el throughput.
-

Reto 3: Reflexión final

Hot partitioning con 6 VIPs

Si NVT-VIP-1 a NVT-VIP-6 generan el 80% del tráfico, y el hash los pone en distintas particiones, esas 6 particiones quedan saturadas mientras las demás están ociosas.

Soluciones:

1. **Particionador custom**: detecta claves "VIP" y las distribuye uniformemente ignorando el hash.
2. **Cambiar el schema de claves**: concatenar `VIP_ID + bucket(timestamp, 10)` para distribuir cada VIP en 10 sub-claves.
3. **Tópico separado para VIPs**: aislar el tráfico de alta carga en su propio tópico con más particiones.

Hot partitioning con vehículos comunes

Si tienes 5000 vehículos con tráfico parejo, el hash distribuye uniformemente y no hay problema. El issue solo aparece cuando hay **skew** en la generación de tráfico.

Cuándo no importa el orden por clave

- **Métricas agregadas**: contar eventos totales por minuto. El orden de llegada de eventos individuales no afecta la suma.
- **Logs de aplicación**: cada evento es independiente; el orden ayuda al debugger pero no es estricto.
- **Procesamiento por batch**: si procesas cada hora todos los eventos del último minuto, el orden dentro del batch no afecta el resultado.
- **Eventos con timestamp**: si el receptor reordena por timestamp, no necesitas que Kafka lo mantenga.

Cuándo Sí importa el orden por clave

- **Estado de entidades**: actualizar el estado de un vehículo (MOVING → STOPPED → MAINTENANCE). Si llegan en otro orden, el estado final puede ser incorrecto.
 - **Pagos / saldos**: depósitos y retiros que afectan un saldo. El orden importa.
 - **Eventos de máquinas de estado**: cualquier sistema con transiciones que dependen del estado anterior.
-

LAB 07

Consumer groups bajo presión

Lab 07 — Reporte resuelto (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearte con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Parte 1: Estrategias de asignación

Comportamiento esperado con 12 particiones y 3 consumers

Estrategia	Asignación esperada
Range	Consumer 1: [0,1,2,3], Consumer 2: [4,5,6,7], Consumer 3: [8,9,10,11]
RoundRobin	Consumer 1: [0,3,6,9], Consumer 2: [1,4,7,10], Consumer 3: [2,5,8,11]
Sticky	Similar a RoundRobin, pero al rebalancear MANTIENE la asignación previa
CooperativeSticky	Igual a Sticky pero con rebalanceo INCREMENTAL (los no afectados siguen procesando)

Sticky agregando un 4to consumer

- **Antes (3 consumers):** ~4 particiones por consumer
- **Después (4 consumers):** 3 particiones por 3 de ellos, 3 por el cuarto
- **Particiones movidas:** típicamente 2-3 (las que se ceden al nuevo). Sticky minimiza el movimiento.

Por qué importa minimizar re-asignación

- Cada re-asignación implica **commit offsets, descartar batches en memoria, re-fetch desde brokers**.
- Si una partición cambia de dueño, el nuevo consumer debe esperar al commit previo y empezar desde ahí.

- En producción con miles de mensajes/segundo, mover muchas particiones = pausa larga y duplicado-procesamiento si commits no estaban al día.

Parte 2: Lag

Métricas típicas

- **LAG inicial (2 consumers, sin carga):** 0 o muy bajo (los consumers procesan al instante).
- **Tras flood de 50K:** lag total sube rápido a 30K-50K mientras los 2 consumers hacen catch-up.
- **Con 6 consumers:** el lag baja en ~30-90 segundos.
- **Con 14 consumers vs 12 particiones:** 2 consumers idle. Es desperdicio.

Parte 3: Rebalanceo

Métricas típicas

Estrategia	Tiempo rebalanceo	Stop-the-world
Eager (Range)	5-15 segundos	Sí, todos los consumers paran
CooperativeSticky	2-5 segundos	No, los consumers no afectados siguen procesando

Recomendación

CooperativeSticky para producción. Eager solo en clústers muy chicos donde el overhead del cooperative no compensa.

Parte 4: Offsets

Reset por timestamp

- Cambia CURRENT-OFFSET a la posición del primer mensaje con timestamp $\geq$ TS.
- Requiere consumers inactivos: si hay un consumer activo, el coordinator no puede tomar control de los offsets sin causar inconsistencias.

Reset a offset específico

- Solo afecta la partición especificada.
- Las demás permanecen igual.

Skip poison message

- Sube el offset en +1.
 - **Problema:** el mensaje se pierde sin trazabilidad. No sabes qué tenía, por qué falló, ni cómo evitar que vuelva a pasar.
 - **Solución:** DLQ (Parte 5).
-

Parte 5: Dead Letter Queue

Comportamiento esperado

- **[OK]:** 90 mensajes
- **[DLQ]:** 10 mensajes
- **DLQ topic:** 10 mensajes
- **Información perdida:** headers, timestamp original, partición, offset, key (si los hubiera).

Reflexión

- **Por qué DLQ mejor que skip:** preserva el mensaje para análisis post-mortem; permite re-procesarlo después de fixearlo; permite alertar (si DLQ crece, hay un bug).
 - **Si el procesador DLQ falla:** tener una "DLQ de la DLQ" es una opción extrema; lo más sano es alertar al equipo y pausar el flujo.
 - **Loop infinito:** usar header `failure_count` y descartar (o mover a "morgue") tras N reintentos.
 - **Retención DLQ:** típicamente más larga que el tópico principal (30-90 días) para dar tiempo a investigar.
-

Solución - Lab 07

Lab 07 — Respuestas del desafío (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Reto 1-3: DLQ en acción

Resultado esperado

Con 100 mensajes (10 con `POISON`, 90 normales): - 90 mensajes con `[OK]` (procesados) - 10 mensajes con `[DLQ]` (redirigidos a `novatech.lab07.dlq`) - Verificación: el tópico DLQ contiene exactamente los 10 mensajes venenosos.

Información perdida en este wrapper simplificado

- **Headers** del mensaje original (cualquier metadata custom).
- **Partición** original (a qué partición del tópico principal correspondía).
- **Offset** original (en qué posición estaba).
- **Timestamp** del producer original.
- **Key** del mensaje (si la tenía).

En producción profesional, la DLQ guardaría todo esto en headers del nuevo mensaje: -

`original.topic` - `original.partition` - `original.offset` - `original.timestamp` -
`error.reason` - `failure.count`

Reto 4: Reflexión

¿Por qué DLQ mejor que skip?

1. **Preserva evidencia:** el mensaje sigue existiendo, alguien puede investigarlo.
2. **Permite re-procesamiento:** si el bug del consumer se fixa, se puede consumir desde la DLQ.
3. **Permite alertas:** si la DLQ crece, hay un problema sistémico que merece atención.
4. **No pierde el negocio:** si el mensaje era un pago real, saltarlo es perder dinero.

Si el procesador de la DLQ también falla

Opciones de menor a mayor radicalidad:

1. **Alertar y pausar**: notificar al equipo y detener el flujo en la DLQ.
2. **DLQ-de-la-DLQ**: tópico para los mensajes que ni siquiera la DLQ pudo procesar.
3. **Quarantine permanente**: a partir de N fallos, mover a un tópico "morgue" que nadie procesa, solo se inspecciona manualmente.

En la práctica, **la opción 1 es la sana**. Las opciones 2-3 esconden el problema.

Cómo evitar el loop infinito DLQ→reintento→DLQ

- **Header `failure_count`**: cada vez que un mensaje falla, se incrementa.
- **Política**: si `failure_count > N` (típicamente 3-5), el mensaje no se re-encola; se manda a "morgue" o se alerta.
- **Backoff exponencial**: entre reintentos, aumentar el delay (1s, 2s, 4s, 8s...) para no saturar.

Retención recomendada para DLQ

- **Tópico principal**: 7 días default.
- **DLQ**: 30-90 días. Razones:
 - Da tiempo al equipo para investigar.
 - Permite re-procesar cuando se fixa el bug.
 - Costo de almacenamiento es bajo (volumen de DLQ << volumen principal).
- **Compactación**: NO usar `cleanup.policy=compact` en DLQ; cada mensaje fallido es único e importante.

LAB 08

Monitoreo con Control Center

Lab 08 — Reporte resuelto (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Parte 1: Arquitectura

Respuestas esperadas

Pregunta	Respuesta esperada
2 UIs cargaron	Sí (Control Center 9021, Kafbat 8090)
Brokers que ve CC	3
Imagen kafka-broker-1	<code>confluentinc/cp-server:7.9.0</code>
¿Por qué cp-server?	Es la versión enterprise que incluye <code>ConfluentMetricsReporter</code> . Sin esto, CC no recibiría métricas y mostraría dashboards vacíos
Variables clave	<code>KAFKA_METRIC_REPORTERS=io.confluent.metrics.reporter.ConfluentMetricsReporter</code> y <code>KAFKA_CONFLUENT_METRICS_REPORTER_BOOTSTRAP_SERVERS</code>
Dónde se almacenan las métricas	En el tópico interno <code>_confluent-metrics</code> del propio clúster Kafka
¿Existe <code>_confluent-metrics</code> ?	Sí, se crea automáticamente al arrancar los brokers

Reflexión

- **CC caído:** los brokers siguen normalmente. Las métricas se siguen acumulando en `_confluent-metrics`. Cuando CC vuelva, recupera el histórico desde el tópicos (con la retención configurada).
 - **Brokers caídos:** CC pierde la fuente de métricas. Pero los datos previos quedan en el tópicos hasta que se purguen por retención.
 - **cp-server vs cp-kafka:** solo `cp-server` viene con `ConfluentMetricsReporter` empaquetado. `cp-kafka` es la versión open-source pura sin extensiones enterprise.
-

Parte 2: Tour Control Center

Métricas típicas

- **Brokers:** 3 (todos UP)
- **Tópicos:** ~10-15 (usuario + internos `_confluent-*`, `__consumer_offsets`)
- **Mensajes/seg:** depende de carga; con productor GPS solo, ~0.5 msg/seg

Distribución de líderes

Con 3 brokers y RF=3, las particiones se distribuyen uniformemente. Para `novatech.lab08.transactions` (12 particiones), cada broker debería ser líder de 4.

Active Controller

Varía: cualquiera de los 3 (el primero electo en KRaft). Generalmente el broker-1 al ser el primero en arrancar.

Tópicos `_confluent-*` esperados

- `_confluent-metrics`: las métricas que CC consume
 - `_confluent-controlcenter-*`: tópicos internos de CC para su estado
 - `_confluent-monitoring`: monitoreo de interceptors (si aplica)
 - `_confluent-license`: gestión de licencia
-

Parte 3: Métricas bajo carga

Métricas típicas con `produce-flood.sh 600 200`

- **Production rate:** ~200 msg/seg (puede aparecer ligeramente menor por agregación)
- **Throughput:** ~40 KB/seg (200 msg × 200 bytes)
- **Consumo:** depende; los consumers internos de CC consumen tópicos `_confluent-*`

Distribución entre particiones

Esperado: las 12 particiones reciben mensajes de manera relativamente uniforme (sticky partitioner del producer-perf-test).

Comparación Kafbat vs CC

- **Kafbat**: vista en tiempo real instantánea, pero sin histórico.
- **CC**: dashboards con tendencia histórica desde que CC arrancó (lee de `_confluent-metrics` que tiene retención).

Diferencia clave: **CC permite ver "qué pasó hace 1 hora", Kafbat solo "qué pasa AHORA".**

Parte 4: Alerta

Tiempos típicos

- **Detección**: 30-90 segundos después de tumbar el broker
- **Resolución**: 30-90 segundos después de revivir el broker

Por qué duraciones

- **30s no es instantáneo**: el `ConfluentMetricsReporter` publica al tópico cada 60s por default; CC lee y evalúa la regla.
- **Duration=1s**: causaría flapping. Cualquier blip momentáneo dispararía la alerta.
- **Producción**: típicamente 5 min para alertas no críticas, 30s-2min para críticas.

Verificación

- **CC > Alerts**: alerta activa en rojo en la página de alertas
- **CC > Brokers**: el broker tumbado aparece como down
- **CC > Topics**: tópicos con réplicas faltantes muestran warnings

Parte 5: Desafío

Tabla comparativa esperada

Aspecto	Kafbat UI	Control Center
Costo	Gratis (Apache 2.0)	Comercial (eval gratis)
Tamaño imagen	~200 MB	~1.5 GB
Métricas históricas	No	Sí (vía <code>_confluent-metrics</code>)

Aspecto	Kafbat UI	Control Center
Dashboards	Básicos	Profesionales
Alertas configurables	No	Sí (nativas en CC)
RBAC	No (en open source)	Sí
Stream Designer	No	Sí
Producir mensajes desde UI	Sí	Sí
Curva aprendizaje	Baja	Media-alta

Decisión por caso

- **Startup 1 dev:** Kafbat. Suficiente, gratis, rápido.
- **Empresa 50 clústers:** CC. Necesitas RBAC, alertas, dashboards profesionales.
- **Lab aprendizaje:** Kafbat. Más simple para empezar.
- **Producción crítica:** CC. Alertas son no-negociables.
- **Investigación rápida:** Kafbat. Más ágil.
- **Reporte ejecutivo:** CC. Dashboards "presentables".

Conclusión esperada

Muchas empresas usan AMBAS: Kafbat para devs y CC para NOC/operaciones. No es excluyente.

Solución - Lab 08

Lab 08 — Respuestas del desafío (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearte con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Reto 1-2: Comparativa y casos de uso

Mensaje pedagógico clave

Kafbat UI y Control Center NO compiten directamente. Son herramientas para casos de uso distintos:

- **Kafbat UI** = "navegador" del clúster: ver tópicos, mensajes, configs, consumer groups. Lightweight, gratis, rápido.
- **Control Center** = "centro de operaciones": dashboards históricos, alertas, RBAC, integración con Schema Registry y Connect.

Tabla detallada

Aspecto	Kafbat UI	Control Center
Filosofía	"Browse and inspect"	"Operate and monitor"
Audiencia	Devs, SREs ágiles	NOC, gerencia, equipo de operaciones
Despliegue	1 contenedor, 200 MB	1 contenedor (1.5 GB), CC Legacy autocontenido
Costo	Gratis (open source Apache 2.0)	Eval gratis, comercial para producción
Aprendizaje	<30 min	1-2 días para dominar

Reto 3: Limitaciones honestas

Limitaciones de Kafbat

- **No tiene retención de métricas:** solo muestra "ahora". Si un broker tuvo problema hace 1 hora, no puedes verlo.
- **No tiene alertas:** si nadie está mirando la UI, nadie se entera del problema.
- **No tiene RBAC nativo en versión open:** cualquiera con acceso ve todo.

- **No tiene integración con Schema Registry visual.**

Limitaciones de Control Center

- **Pesado:** 1.5 GB de imagen, 1-2 GB de RAM en uso normal.
- **Tarda en arrancar:** 60-90 segundos. En desarrollo es molesto.
- **License para producción:** el período de evaluación está bien para POCs y aprender, pero para uso prolongado hay que pagar.
- **Curva de aprendizaje:** la UI es densa, hay muchas pestañas.
- **Dependencia del ecosistema Confluent:** se integra naturalmente con Schema Registry, Connect, ksqldb, pero NO con herramientas externas de Apache (ej. Kowl, Conductor).

Reto 4: Decisiones reales

Empresa de 5 personas

Kafbat UI. Razones: - Gratis - Rápido de levantar - Suficiente para "ver qué pasa" - No necesitas RBAC con 5 personas (todos son trusted) - Si un broker falla, lo notarán de inmediato sin alertas formales

Empresa de 500 personas

Control Center (probablemente con Kafbat también para devs). Razones: - RBAC obligatorio (no todos pueden ver todo) - Alertas automáticas son críticas (nadie está mirando la UI 24/7) - Dashboards históricos para análisis post-incidente - Reportes ejecutivos al directorio - Integración con Schema Registry, Connect, etc.

Cuándo usar AMBAS

Configuración profesional muy común: - **Kafbat** instalada en el namespace de "playground"/"sandbox" para que devs experimenten libremente. - **Control Center** instalada en el namespace de "production" con RBAC estricto, conectada a herramientas de incident management (PagerDuty, Slack).

Esto da: - Velocidad de desarrollo (Kafbat) - Robustez de producción (CC) - Sin compromiso entre ambos

Reflexión final

Aspecto	Aprendizaje
Monitoreo es un sistema, no una UI	Brokers + tópico de métricas + UI + alertas trabajan juntos

Aspecto	Aprendizaje
Open source vs Enterprise	Cada uno tiene su lugar. La gratis no es "peor"; es distinta
Alertas > Dashboards	Un dashboard sin alertas = nadie se entera de los problemas
Test bajo carga	Los problemas reales aparecen con carga real, no en clúster vacío

Soluciones del desafío - Lab 08

LAB 09

Kafka Connect

Lab 09 — Reporte resuelto (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Parte 1: Arquitectura

Pregunta	Respuesta esperada
Versión Connect	8.2.0 (corresponde a Kafka 4.2)
<code>kafka_cluster_id</code>	el valor configurado en el broker (ej <code>novatech-lab09-cluster-9091</code>)
<code>JdbcSourceConnector</code>	Sí (instalado vía confluent-hub)
<code>JdbcSinkConnector</code>	Sí
Tópicos <code>_connect-*</code>	<code>_connect-configs</code> , <code>_connect-offsets</code> , <code>_connect-status</code>
Para qué sirve cada tópico	configs = definiciones de connectors; offsets = posición de cada source/sink; status = estado de tasks
Cluster en Kafbat UI	Sí, aparece como <code>connect-novatech</code>

Parte 2: Source connector

Estado típico

- `connector.state` : RUNNING
- Tasks: 1 (porque `tasks.max=1`)
- `tasks[0].state` : RUNNING

Captura

- 5 pedidos seed leídos
- Formato: JSON sin schema (`schemas.enable=false`)
- Campos: `id` , `cliente_id` , `producto` , `cantidad` , `monto` , `estado` , `creado_en`
- Latencia INSERT → Kafka: ~5s (por `poll.interval.ms=5000`)

Reflexión

- **Connect cae a la mitad:** al reiniciar lee `_connect-offsets` para saber el último id procesado. Continúa sin duplicar.
- `mode: incrementing` **solo INSERT:** porque solo detecta nuevos id. UPDATE no cambia id; DELETE saca filas.
- **Capturar UPDATE/DELETE:** usar Debezium (lee el WAL de PostgreSQL).

Parte 3: Sink connector

Pregunta	Respuesta esperada
Tabla vacía al inicio	Sí (init-novatech.sql crea sin INSERTs en pedidos_procesados)
Estado tras crear	RUNNING
Mensaje id=1 aparece	Sí, ~3-5s después
Re-publicar id=1	NO se duplica, se actualiza (upsert)
Por qué	<code>insert.mode: upsert</code> + <code>pk.fields: id</code>

Mensaje malformado

- Estado: FAILED
- Error: típicamente `DataException: ... missing required field 'id'` o similar
- Recuperación: `curl -X POST http://localhost:8083/connectors/novatech-sink-procesados/restart`

Reflexión

- `auto.create: false` : para tener control sobre tipos de columna. Auto-create usa tipos genéricos.
- **Upsert vs insert:** upsert para idempotencia; insert puro si nunca habrá duplicados de PK.
- **Sink atrasado:** los mensajes esperan en Kafka. Cola natural sin pérdida.

Parte 4: Desafío

Flujo end-to-end

1. INSERT en pedidos → Source captura en ~5s → publica a Kafka
2. Consumer ve el mensaje
3. Cliente publica al tópico `pedidos.procesados`
4. Sink lo recoge en ~3-5s → INSERT/UPDATE en `pedidos_procesados`

Tiempo total típico: 10-15 segundos.

Reflexión

- **Líneas de código:** cero (solo configs JSON)
- **Vs Python:** Connect maneja resiliencia (reinicia tasks fallidas), paralelismo, offsets, reintentos. En Python tendrías que codificar todo eso.
- **Connect cae a la mitad:** los offsets están en `_connect-offsets`. Al reiniciar, continúa donde dejó. No pierde mensajes ni duplica.
- **Diferencia con Debezium:** JDBC source con `incrementing` solo captura INSERTs. Debezium lee el WAL (Write-Ahead Log) de PostgreSQL y captura INSERT/UPDATE/DELETE en orden, con captura de cambios en tiempo casi real (ms vs segundos).
- **Otros conectores útiles:** S3 Sink (archivar), Elasticsearch Sink (indexar para búsqueda), Snowflake Sink (data warehouse), MongoDB Source (NoSQL), Salesforce Source (CRM), HTTP Source/Sink (APIs).

Solución - Lab 09

Lab 09 — Respuestas del desafío (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Reto 1-4: Flujo end-to-end

Comportamiento esperado

Los 5 pedidos insertados aparecen en Kafka en ~5 segundos. Los 5 mensajes "procesados" aparecen en `pedidos_procesados` en otros ~5 segundos. Tiempo total: ~10-15s.

Tabla esperada después del flujo

```
SELECT id, cliente_id, producto, estado, procesado_en
FROM pedidos_procesados
ORDER BY procesado_en DESC LIMIT 10;
```

5 registros con IDs 6-10 (suponiendo que arrancaste con 5 seed + 5 desafío).

Reto 5: Reflexión final

Líneas de código escritas: CERO

Toda la lógica está en: - `jdbc-source-pedidos.json` : 17 líneas de config JSON - `jdbc-sink-procesados.json` : 18 líneas de config JSON

Total: 35 líneas de **configuración declarativa**, no código.

Vs implementación con Python

Para hacer lo mismo en Python necesitarías: 1. Cliente de PostgreSQL (psycopg2) 2. Cliente de Kafka (confluent-kafka-python) 3. Lógica de polling de la tabla con tracking del último id visto 4. Persistencia del último id (en archivo o tabla) 5. Manejo de errores: timeouts, reconexión, retries 6. Logging 7. Health checks 8. Despliegue (Dockerfile, supervisord, etc.) 9. Misma cosa para el flujo Kafka → PostgreSQL

Probablemente **500-1000 líneas de código** + tests + ops.

Connect te da TODO eso "gratis" a cambio de aprender la sintaxis declarativa.

Si Connect cae a la mitad

- Los offsets están en `_connect-offsets` (tópico replicado en 3 brokers).
- Al reiniciar Connect, lee el último offset procesado y continúa desde ahí.
- En modo distributed con múltiples workers, las tasks se redistribuyen automáticamente.
- **No pierde mensajes ni duplica** (assuming idempotencia del sink, que el JDBC Sink con upsert garantiza).

Diferencia con Debezium

Aspecto	JDBC connector (este lab)	Debezium
Modo de captura	Polling (cada 5s)	Streaming del WAL
Latencia	~5 segundos	<100 ms
Captura UPDATE	NO (con incrementing)	Sí
Captura DELETE	NO	Sí
Carga en DB	Query SELECT cada 5s	Lee log de transacciones
Configuración	Más simple	Requiere replicación de PostgreSQL
Caso de uso	Demos, datos con alta latencia tolerable	CDC real-time, productos críticos

Regla: para producción seria, usar Debezium. Para introducción pedagógica, JDBC.

Otros conectores útiles para NovaTech

- **S3 Sink:** archivar todos los pedidos a S3 para compliance/auditoría
- **Elasticsearch Sink:** indexar pedidos para búsqueda full-text
- **MongoDB Source/Sink:** si NovaTech tiene catálogo de productos en MongoDB
- **HTTP Sink:** notificar APIs externas (ej. confirmación al cliente)
- **Salesforce Source:** si hay datos de CRM
- **Twilio/SendGrid Sink:** SMS/email cuando se procese un pedido
- **Snowflake Sink:** enviar al data warehouse para analytics

Confluent Hub tiene 200+ conectores oficiales.

Reto 6: Inspección con Kafbat UI

Kafbat UI > Connect muestra: - Lista de conectores con estado (RUNNING, FAILED, etc.) - Configuración de cada uno - Tasks y su distribución - Permite reiniciar/eliminar conectores desde la UI

Útil para operación diaria sin tener que recordar comandos `curl`.

Soluciones del desafío - Lab 09

LAB 10

Schema Registry y ksqlDB

Lab 10 — Reporte resuelto (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Parte 1: Schema Registry

Respuestas esperadas

Pregunta	Respuesta esperada
Subjects al inicio	<code>[]</code>
ID schema v1	Un entero, típicamente 1
Versión tras v1	1
¿v2 compatible?	Sí (<code>{"is_compatible": true}</code>)
Versión tras v2	2
¿v3 compatible?	NO (<code>{"is_compatible": false}</code>)
Por qué v3 NO	Agrega campo OBLIGATORIO sin default. Un consumer leyendo datos producidos con v1 no encontraría <code>tarjeta_credito</code> y no sabe qué default usar. Rompe BACKWARD compatibility
Código HTTP error	409 Conflict (Incompatible schema)

Reflexión

- **Sin SR:** cada equipo coordina manualmente; cualquier cambio en formato requiere comunicarse con TODOS los consumers. SR centraliza el contrato.
- **Cuándo FORWARD:** cuando consumers viejos NO se pueden actualizar (mainframes legacy, apps de terceros). Permite agregar campos siempre que tengan default razonable.
- `_schemas` **tópico:** SR usa Kafka como su backing store. Mismas garantías de durabilidad y replicación que los datos.

Parte 2: Avro

Respuestas esperadas

Pregunta	Respuesta esperada
Pedido Avro publicado	Sí
Aparece en consume-avro como JSON	Sí (kafka-avro-console-consumer deserializa)
5 mensajes en Kafbat UI	Sí
Kafbat los muestra como JSON	Sí (integración con SR)
Throughput flood 50	Variable, ~5-30 msg/seg en local
4 clientes publicados	Sí

Reflexión

- **Avro vs JSON:** Avro ~30-50% del tamaño de JSON equivalente. La razón: schema separado, no se repiten nombres de campos en cada mensaje.
- **Por qué a gran escala:** menos bytes en red + disco + memoria. A 1M msg/seg, la diferencia es enorme.
- **Monto como string:** el producer Avro lo rechaza ANTES de publicar, validando contra el schema. Sin SR, llegaría al tópico y rompería consumers downstream.

Parte 3: ksqldb

STREAM y TABLE

- `SHOW STREAMS` muestra `PEDIDOS_STREAM` (UPPERCASE).
- `EMIT CHANGES`: aparecen pedidos en tiempo real, latencia ~1-3s.
- `WHERE monto > 50000`: solo aparece el pedido cumpliendo el filtro (75000), no el de 5000.
- TABLE necesita PRIMARY KEY porque materializa estado: sin key no sabe qué fila reemplazar.

- Con 2 mensajes mismo key, la TABLE muestra solo el ÚLTIMO.

Reflexión

- **STREAM vs TABLE:** STREAM = eventos (logs, transacciones); TABLE = estado actual (perfiles, configs).
 - **EMIT CHANGES:** push query continuo. Sin él, ksqldb devuelve un snapshot puntual (pull query, 0.x+).
-

Parte 4: Desafío

Reto 1: Filtro persistent

- Crea `PEDIDOS_ALTO_VALOR` que es un STREAM derivado.
- `SHOW QUERIES` muestra la persistent query con un ID tipo `CSAS_PEDIDOS_ALTO_VALOR_0`.
- De los 2 pedidos: solo aparece el de 99999 (cumple `monto > 50000`); el de 1000 NO.

Reto 2: Agregación con ventana

- Aparecen filas `cliente_id, count, sum` por ventana de 1 minuto.
- Al cambiar de minuto: aparecen NUEVAS filas (la ventana anterior se "cierra").
- Es agregación POR ventana, no acumulativa eterna.

Reto 3: JOIN stream-table

- Pedido 200 (cliente 1001 existente): `cliente_nombre = "Acme S.A. - actualizado"`, `cliente_tipo = VIP`, etc.
- Pedido 201 (cliente 9999 inexistente): `cliente_nombre = NULL`, etc. Con LEFT JOIN aparece igual; con INNER JOIN no.

Reto 4: Filtrar VIPs

- Aparece pedido 300 (cliente 1001 = VIP).
- NO aparece pedido 301 (cliente 1002 = ESTANDAR).

Reflexión final

- **Líneas de Java ahorradas:** cada query persistent equivale a ~200-500 líneas Kafka Streams. 4 queries = 1000-2000 líneas no escritas.
- **Reiniciar ksqldb:** NO se pierde nada. Los streams/tables son metadatos en `_confluent-ksql-novatech_command_topic`. ksqldb los redescubre y reinicia las queries.
- **Cuándo NO usar ksqldb:** lógica imperativa compleja (loops, condiciones complicadas), llamadas HTTP a APIs externas, custom serializers/deserializers, algoritmos de ML. En esos casos, Kafka Streams (Java) o aplicaciones custom.

- **Bajo el capó:** cada persistent query es una aplicación Kafka Streams compilada al vuelo. Tiene sus propios offsets, state stores, etc.
-

Solución - Lab 10

Lab 10 — Respuestas del desafío (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Streaming SQL en producción

Este desafío demuestra capacidades que en producción real reemplazan a aplicaciones Kafka Streams completas.

Reto 1: Persistent query

```
CREATE STREAM pedidos_alto_valor AS
  SELECT id, cliente_id, producto, monto, estado
  FROM pedidos_stream
  WHERE monto > 50000;
```

Lo que pasa por debajo: 1. ksqlDB compila esta query a una aplicación Kafka Streams. 2. Crea un nuevo tópico Kafka (`PEDIDOS_ALTO_VALOR`) con el resultado. 3. Lanza una task que lee de `pedidos_stream`, filtra, y publica al nuevo tópico. 4. La task corre **indefinidamente** hasta `TERMINATE` o `DROP STREAM`.

Para verificar el tópico creado:

```
kafka-cli/list-topics.sh | grep PEDIDOS
```

Reto 2: Agregación con ventana

```
SELECT cliente_id, COUNT(*) AS total_pedidos, SUM(monto) AS suma_montos
FROM pedidos_stream
WINDOW TUMBLING (SIZE 1 MINUTE)
GROUP BY cliente_id
EMIT CHANGES;
```

TUMBLING window: ventanas no solapadas, fijas, contiguas. Ejemplo: - 14:00-14:01 → ventana 1 - 14:01-14:02 → ventana 2 - 14:02-14:03 → ventana 3

Para cada cliente_id, ksqldb cuenta cuántos pedidos hizo en CADA ventana. Si un cliente hizo 5 pedidos en la ventana 1 y 3 en la 2, aparecen 2 filas distintas.

Otras ventanas: - **HOPPING**: solapadas (ej. ventanas de 5 min cada 1 min) - **SESSION**: agrupa eventos cercanos en el tiempo

Reto 3: JOIN stream-table

```
CREATE STREAM pedidos_enriquecidos AS
  SELECT p.id, p.producto, p.monto, c.nombre, c.tipo, c.ciudad
  FROM pedidos_stream p
  LEFT JOIN clientes_table c ON p.cliente_id = c.id;
```

Bajo el capó: 1. Para cada pedido entrante, ksqldb busca en `clientes_table` por la key. 2. Si encuentra, agrega los campos del cliente al evento de pedido. 3. Si NO encuentra (LEFT JOIN), agrega NULL. 4. Publica al nuevo tópico `PEDIDOS_ENRIQUECIDOS`.

Diferencia LEFT vs INNER JOIN: - LEFT: incluye pedidos aunque el cliente no exista (NULLs). - INNER: solo incluye pedidos cuyo cliente existe en la TABLE.

Importante: la TABLE debe tener datos ANTES de que lleguen los pedidos. Si publicas primero un pedido y luego el cliente, ese pedido se procesará con cliente NULL aunque después aparezca el cliente.

Reto 4: Filtrar VIPs

```
SELECT pedido_id, producto, monto, cliente_nombre, cliente_ciudad
FROM pedidos_enriquecidos
WHERE cliente_tipo = 'VIP'
EMIT CHANGES;
```

Esta query es un **push query** sobre el stream `pedidos_enriquecidos`. NO crea un nuevo stream persistente; solo muestra resultados al cliente conectado.

Si quisieras alertar VIPs persistentemente:

```
CREATE STREAM alertas_vip AS
  SELECT * FROM pedidos_enriquecidos WHERE cliente_tipo = 'VIP';
```

Reto 5: Reflexión final

Líneas de Java ahorradas

Cada persistent query ksqlDB equivale a: - Filtro simple: ~50 líneas Kafka Streams - Agregación con ventana: ~150 líneas - JOIN: ~200 líneas

4 queries totales: ~**500-700 líneas de Java NO escritas**, ni testeadas, ni desplegadas.

Reiniciar ksqlDB

ksqlDB persiste sus metadatos (streams, tables, queries) en el tópico interno `_confluent-ksql-novatech_command_topic`. Al reiniciar: 1. Lee ese tópico. 2. Recrea los streams/tables. 3. Vuelve a lanzar las persistent queries.

No se pierde nada, solo se interrumpe el procesamiento durante el reinicio (segundos).

Cuándo NO usar ksqlDB

Caso	Por qué NO
Lógica imperativa compleja	SQL es declarativo; bucles complejos requieren código
Llamadas a APIs HTTP externas	ksqlDB no soporta side-effects
Custom serializers/deserializers	El SDK de Kafka Streams te da más control
Algoritmos ML / AI	Necesitas librerías Java/Python
Latencia ultra baja (<10ms)	ksqlDB tiene overhead vs código optimizado

En esos casos, **Kafka Streams en Java** es la respuesta.

Persistent query bajo el capó

Cada `CREATE STREAM ... AS SELECT ...` o `CREATE TABLE ... AS SELECT ...`:

1. ksqlDB **parsea el SQL** y lo convierte a un plan lógico.
2. Compila el plan a una aplicación **Kafka Streams**.
3. La aplicación tiene su propio: - **Application ID** (basado en `service.id` + nombre query) - **State stores** (RocksDB local) para agregaciones y joins - **Consumer group** (offsets en `__consumer_offsets`)
4. Procesa eventos en tiempo real, escribe al tópico destino.
5. Tolerante a fallos: state stores backup en tópicos `*-changelog`.

Es decir: **ksqlDB es un compilador SQL → Kafka Streams**. Si miras `docker logs ksqldb-server`, verás logs de aplicaciones Kafka Streams arrancando.

Soluciones del desafío - Lab 10

LAB 11

Prometheus y Grafana

Lab 11 — Reporte resuelto (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Parte 1: Arquitectura

Pregunta	Respuesta esperada
3 endpoints JMX	Sí, los 3 responden con métricas Prometheus
Tipos	counter (acumulados, ej <code>_total</code>), gauge (instantáneos, ej <code>_per_second</code>), eventualmente histogram
Targets totales	4 (1 prometheus self + 3 brokers)
Targets UP	4 (todos sanos al inicio)
Dashboard Grafana	Sí, "Kafka Cluster Overview" pre-cargado

Parte 2: Tour Grafana

Métricas iniciales (clúster sin carga)

- Bytes In/Out: ~0 (puede haber tráfico mínimo de tópicos internos)
- Request Rate: bajo, principalmente FetchFollower (replicación interna)
- URP: 0
- Active Controllers: 1 (el primero electo en KRaft)

Por qué Active Controllers = 1

- 0 → clúster sin liderazgo, NO acepta cambios de metadata (crear tópicos, etc.)
- 2 → "split-brain", catástrofe operacional
- 1 → estado normal: exactamente un controller activo

Paneles "No data" típicos

El dashboard 11962 puede tener queries para métricas que requieren: - Otras versiones de Kafka - JMX Exporter con reglas distintas - Métricas de ZooKeeper (ya no aplica con KRaft)

Es **típico** y enseñable: dashboards de la comunidad son punto de partida.

Parte 3: Métricas bajo carga

Con flood 600s @ 200 msg/s

- Bytes In total: ~40 KB/s (200 × 200 bytes)
- Distribución entre brokers: uniforme aproximadamente (sticky partitioner)
- Throughput total: ~200 msg/seg (coincide con producido)

Tras tumbar broker 2

- Active Controllers: puede pasar de 1 a 0 brevemente y luego volver a 1 (re-elección)
- URP: sube a varios (todas las particiones cuyo follower era broker 2)
- Bytes In/Out broker 2: cae a 0
- Targets UP: baja a 3

Recuperación

- Broker 2 vuelve UP en ~30s
- URP vuelve a 0 en 30-60s después (catch-up)

Parte 4: Confluent Cloud

Pregunta	Respuesta
¿Tour o práctico?	Demostrativo (instructor muestra)
Crédito gratis	\$400 USD por 30 días
Tipos cluster	Basic, Standard, Dedicated, Enterprise
¿Por qué no práctico?	Requiere tarjeta de crédito personal; en cursos corporativos genera fricción

Pregunta	Respuesta
Latencia	10-50 ms vs <1ms local
Ventajas Cloud	SLA 99.99%, escalabilidad elástica, cero ops
Desventajas Cloud	Costo recurrente, latencia de red, datos fuera de tu DC

Parte 5: Comparativa final

Caso	Recomendación
Startup	Kafbat UI + Prometheus/Grafana
Empresa 500 personas	CC o Cloud (RBAC + dashboards profesionales)
Producción crítica con compliance	CC + Prometheus para detalle técnico
¿Usar varias?	Sí, MUY común. Cada una para su público
¿Por qué no técnica?	Costo, equipo, criticidad, compliance pesan más que features

Solución - Lab 11

Lab 11 — Respuestas del desafío (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Sobre el dashboard 11962 de la comunidad

El dashboard "Kafka Cluster Overview" (Grafana ID 11962) tiene queries PromQL que asumen ciertas convenciones de nombres de métricas. Las que JMX Exporter genera con `kafka-broker.yml` pueden no coincidir 1:1.

Esto es normal y enseñable: en operación real, importar dashboards de la comunidad es un buen punto de partida, pero siempre requiere ajustes.

Cómo diagnosticar paneles "No data"

1. Click en el panel → Edit
2. Mira la query PromQL
3. En Prometheus (<http://localhost:9090>) prueba la métrica: `promql kafka_server_brokertopicmetrics_messagesinpersec_total`
4. Si no existe, busca alternativa: `{__name__=~"kafka_server.*"}`
5. Ajusta la query del panel

Métricas que SÍ funcionan con nuestro setup

- `kafka_server_brokertopicmetrics_bytesinpersec_total`
- `kafka_server_brokertopicmetrics_bytesoutpersec_total`
- `kafka_server_brokertopicmetrics_messagesinpersec_total`
- `kafka_server_replicamanager_underreplicatedpartitions`
- `kafka_controller_kafkacontroller_activecontrollercount`

Sobre Confluent Cloud

Por qué tour vs práctico

En cursos corporativos: - Pedir tarjeta de crédito personal a un alumno = problema legal/HR - La cuenta del instructor se reusa en cada sesión - El alumno ve el flujo completo, anota observaciones, replica en casa si quiere

Comparativa real de costos (orientativa)

Carga	Local (Docker)	Cloud (Basic)	Cloud (Standard)
0 mensajes/seg	\$0	\$0 (free tier)	~\$50/mes
1.000 msg/seg	\$0 (luz)	~\$80/mes	~\$200/mes
100.000 msg/seg	Hardware grande	\$1.500-3.000/mes	\$5.000-15.000/mes

Lección: Cloud es razonable para producción; local sigue siendo el rey para dev/staging.

Sobre la comparativa final

Por qué la elección NO es técnica

Kafka es Kafka en cualquier herramienta de monitoreo. Las diferencias son:

1. **Costo:** open-source = \$0 software pero \$\$ ops; managed = \$\$\$ pero cero ops
2. **Equipo:** ¿tienes personas que puedan operar Prometheus/Grafana?
3. **Criticidad:** ¿qué pasa si tu monitoreo cae 1 hora?
4. **Compliance:** algunos sectores (banca, salud) no pueden enviar datos a Cloud
5. **Velocidad de iteración:** managed permite enfocarse en negocio, no en ops

Combinaciones reales que funcionan

[Producción]

- └─ Confluent Cloud (Kafka managed)
- └─ Prometheus + Grafana (métricas detalladas técnicas)
- └─ Datadog/New Relic (correlación con app metrics)

[Desarrollo]

- └─ Docker local con cp-kafka
- └─ Kafbat UI (vista rápida)
- └─ ksqlDB local (queries ad-hoc)

Soluciones del desafío - Lab 11

LAB 12

Seguridad y capstone final

Lab 12 — Reporte resuelto (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearte con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Parte 1: TLS y certificados

Pregunta	Respuesta de referencia
¿Qué algoritmo y tamaño de llave usa el certificado del broker?	RSA 2048 bits (configurado en <code>bin/generate-certs.sh</code> , parámetro <code>-keyalg RSA -keysize 2048</code>)
¿Quién es el issuer y quién el subject del certificado del broker 1?	Issuer: <code>CN=NovaTech-CA-Lab12</code> (el CA que generamos). Subject: <code>CN=kafka-broker-1</code>
¿Por qué el cliente necesita el truststore pero no el keystore en este lab?	El cliente necesita verificar al broker (truststore con CA), pero el broker no le pide certificado al cliente — la identidad del cliente la valida vía SASL, no vía TLS. Si fuera mTLS también necesitaría keystore.
¿Qué cambiaría si quisiéramos mTLS?	Generar keystore por cliente, configurar <code>KAFKA_SSL_CLIENT_AUTH: required</code> en el broker, y agregar <code>ssl.keystore.*</code> a las properties del cliente.

Parte 2: SASL y autenticación

Pregunta	Respuesta de referencia
¿Cuántos usuarios están definidos en el JAAS server-side?	3: admin, app1, app2 (más la credencial inter-broker <code>username/password</code> , que apunta a <code>admin</code>).
¿Qué error apareció al intentar conectar SIN credenciales?	<code>org.apache.kafka.common.errors.SaslAuthenticationException</code> o timeout en el handshake. El broker cierra la conexión porque el listener EXTERNAL exige SASL_SSL.
¿Por qué <code>User:ANONYMOUS</code> está en super.users?	El listener INTERNAL es PLAINTEXT (sin SASL). Todas las conexiones inter-broker se identifican como <code>User:ANONYMOUS</code> . Sin tenerlo en super.users, las ACLs rechazarían la replicación inter-broker.
¿En qué contexto usarías SCRAM en lugar de PLAIN?	Siempre en producción. PLAIN guarda las passwords en texto plano en el JAAS y las transmite (aunque cifradas por TLS, basta que alguien lea el JAAS). SCRAM hashea la password con salt antes de enviarla.

Parte 3: ACLs y autorización

Pregunta	Respuesta de referencia
Salida abreviada de <code>kafka-cli/list-acls.sh</code> :	3 bloques: <code>User:app1</code> con producer+consumer en publico y confidencial; <code>User:app2</code> con consumer en publico únicamente.
¿Qué error mostró <code>app2</code> al intentar leer el confidencial?	<code>TopicAuthorizationException: Not authorized to access topics: [novatech.lab12.confidencial]</code>
Diferencia entre <code>SaslAuthenticationException</code> y <code>TopicAuthorizationException</code>	La primera = falló la identidad (sin credenciales o credenciales malas). La segunda = la identidad fue OK pero el authorizer no tiene una ACL que permita la operación.
Comando para dar acceso a un nuevo <code>app3</code> solo de lectura sobre <code>novatech.lab12.publico</code>	<code>kafka-acls --bootstrap-server kafka-broker-1:9092 --command-config admin.properties --add --allow-principal User:app3 --consumer --topic novatech.lab12.publico --group '*'</code>

Parte 4: min.insync.replicas

Escenario	¿Funcionó produce?	Razón
3 brokers vivos	Sí	ISR=3, supera min.ISR=2
2 brokers vivos, broker-3 caído	Sí	ISR=2, exactamente igual a min.ISR. Cumple
1 broker vivo, broker-2 y broker-3 caídos	No	ISR=1 < min.ISR=2. <code>NotEnoughReplicasException</code>

Pregunta	Respuesta de referencia
¿Qué min.ISR usarías con RF=5?	min.ISR=3 (sigue la regla <code>RF = min.ISR + 1</code> adaptada: tolera 2 fallos simultáneos manteniendo durabilidad mayoritaria). En clusters críticos donde "quórum" matters.
¿Perdiste datos en algún momento al detener brokers?	No. El cluster prefiere bloquear nuevas escrituras (<code>NotEnoughReplicasException</code>) antes que comprometer durabilidad. Los mensajes ya escritos siguen replicados en los brokers vivos.

Parte 5: RBAC

Pregunta	Respuesta de referencia
¿En qué se diferencia un role binding de RBAC de una ACL nativa?	El role binding asigna un rol predefinido (DeveloperRead, ResourceOwner, etc.) que ya empaqueta operaciones; la ACL es atómica (Read, Write, Describe) y debes componerla manualmente. RBAC además acepta <code>Group:</code> y se enchufa al directorio corporativo.
¿Qué problema operacional resuelve RBAC que ACLs no resuelven a escala?	Onboarding/offboarding de usuarios: un cambio en LDAP propaga permisos automáticamente. Con ACLs nativas hay que tocar cada cluster manualmente. También: cobertura unificada Kafka + Schema Registry + Connect + ksqldb.
¿Justificarías RBAC en un cluster con 3 apps y 2 devs?	No. ACLs + IaC (Terraform o un repo con scripts <code>kafka-acls</code>) cubre. RBAC es licencia paga y plomería extra; tiene sentido a partir de decenas de devs / múltiples clusters / compliance.

Parte 6: Capstone integrador

Desafío 1 — pipeline Connect → Kafka

Salida típica: el último pedido aparece en el topic con offset N+1, generalmente en partición 0 o 1 (depende del round-robin si no hay key). Connector debe estar `RUNNING` con tasks `RUNNING`.

Desafío 2 — JOIN en ksqldb

- El STREAM derivado se crea sin errores siempre que las particiones de origen y la TABLE estén co-particionadas.
- Particiones de `pedidos_enriquecidos`: 3 (heredadas).
- Condición del JOIN: co-particionamiento — mismo número de particiones en el STREAM origen y la TABLE, y la misma key.

Desafío 3 — observabilidad en Grafana

Consumer groups esperados: el del Sink connector (Lab 09), el de ksqldb (Lab 10) y los exploradores manuales con `console-consumer`. El group con mayor lag suele ser el del Sink si lo detuviste recientemente. Métrica para broker no disponible: `up{job="kafka-jmx"}` o el conteo de partitions en `kafka_server_replicamanager_underreplicatedpartitions` (si el dashboard la incluye).

Desafío 4 — diseño seguro

Componente	Respuesta de referencia
Lab 09 (Connect + PostgreSQL)	Connect autenticado al broker vía SASL_SSL con su propio user <code>connect-user</code> . ACLs sobre los topics que produce/consume y sobre <code>connect-configs</code> , <code>connect-offsets</code> , <code>connect-status</code> . PostgreSQL con TLS en su lado y user dedicado de solo lectura para el JDBC source.
Lab 10 (Schema Registry + ksqldb)	Schema Registry con HTTPS y autenticación (basic auth o mTLS). ksqldb con SASL_SSL al broker, su propio principal con ACLs sobre topics que crea y consume, incluyendo prefijos <code>_confluent-ksql-*</code> .
Lab 11 (Prometheus + JMX)	JMX exporter sirviendo via HTTPS con auth básica. Prometheus scrapeando con credenciales. Grafana con SSO/LDAP en lugar de admin/admin.
Cluster en general	TLS en TODOS los listeners (incluido controller). SASL/SCRAM en lugar de PLAIN. mTLS para inter-broker. Super.users acotado a 1 admin humano + cuentas de servicio justificadas. Rotación de certs automatizada.

Desafío 5 (opcional)

Si lo logró: `docker network connect novatech-lab09-net ksqldb-server` y luego en ksqldb usar bootstrap `kafka-broker-1:29092` (los nombres de host de la red Lab 09). Verificable con `SHOW STREAMS;` y un `CREATE STREAM` apuntando al topic original.

Reflexión final

Las respuestas son personales, pero patrones esperables: - Lab más difícil suele ser el 06 (transacciones) o el 11 (JMX rules / dashboards). - Concepto que más cuesta: idempotencia + transacciones, o la diferencia AuthN/AuthZ. - Prioridades para producción: replicación adecuada, monitoreo (lag, ISR, under-replicated), backups/disaster recovery, auth + auditoría.

Lab 12 — Respuestas del desafío (solución de referencia)

⚠ **Importante:** estas son las soluciones de referencia del lab. Antes de consultarlas, intenta resolver cada actividad por tu cuenta. El aprendizaje real está en pelearse con el problema. Estas respuestas son para validar tu trabajo o destrabarte después de intentarlo.

Algunos valores específicos (IDs, timestamps, números de partición que actúan como líder) pueden variar entre ejecuciones — lo importante es la consistencia conceptual.

Desafío 1: insertar pedido y verificar que llega al topic

```
# 1. Inserción en PostgreSQL
docker exec postgres psql -U novatech -d novatech_orders \
  -c "INSERT INTO pedidos (cliente_id, producto, cantidad, monto, estado)
      VALUES (5001, 'Capstone test', 2, 199.50, 'pendiente');"

# 2. Verifica que el connector está RUNNING
cd labs/lab-09-kafka-connect
connect-cli/status-connector.sh novatech-source-pedidos

# 3. Consume del topic los últimos mensajes (Ctrl+C cuando veas el pedido nuevo)
kafka-cli/consume-pedidos.sh
```

Por qué funciona: el JDBC Source connector está configurado en modo `incrementing` o `timestamp+incrementing`, así que cada nuevo `id` insertado en `pedidos` se publica al topic en un par de segundos.

Si NO funciona: ver `labs/lab-09-kafka-connect/docs/troubleshooting.md`. Lo más común es que la task del connector esté en `FAILED` (mensaje malformado anterior) — solución:

```
curl -X POST http://localhost:8083/connectors/novatech-source-pedidos/tasks/0/
restart
```

Desafío 2: STREAM con JOIN en ksqlDB

```
-- Conectarse:
-- docker exec -it ksqldb-cli ksql http://ksqldb-server:8088

SET 'auto.offset.reset'='earliest';

CREATE STREAM pedidos_capstone (
  pedido_id INT, cliente_id INT, producto VARCHAR, monto DOUBLE
) WITH (
  KAFKA_TOPIC='novatech.lab10.pedidos',
  VALUE_FORMAT='AVRO',
  PARTITIONS=3
);

CREATE TABLE clientes_capstone (
  cliente_id INT PRIMARY KEY, nombre VARCHAR, vip BOOLEAN
) WITH (
  KAFKA_TOPIC='novatech.lab10.clientes',
  VALUE_FORMAT='AVRO',
  KEY_FORMAT='AVRO'
);

CREATE STREAM pedidos_enriquecidos AS
  SELECT p.pedido_id, p.producto, p.monto, c.nombre, c.vip
  FROM pedidos_capstone p
  INNER JOIN clientes_capstone c ON p.cliente_id = c.cliente_id;

SELECT * FROM pedidos_enriquecidos EMIT CHANGES;
```

Condición clave del JOIN: co-particionamiento. - El STREAM `pedidos_capstone` debe estar particionado por `cliente_id` (la key del JOIN). - La TABLE `clientes_capstone` ya está particionada por su PRIMARY KEY (`cliente_id`). - Ambos topics deben tener el mismo número de particiones (3 en este lab).

Si los topics no están co-particionados, ksqlDB devuelve:

```
Source topic ... is not co-partitioned with ...
```

Y la solución es hacer un `PARTITION BY cliente_id` antes del JOIN para reparticionar.

Desafío 3: lag y métricas en Grafana

```
# Abrir
open http://localhost:3000 # admin/admin
```

Panels esperados (del dashboard custom del Lab 11): - "Brokers UP" — debe mostrar 3 si el cluster está sano. - "Bytes in/out por broker" — debe haber actividad si produces/consumes. - "Consumer lag por grupo" — el group del Sink connector aparecerá aquí. - "Under-replicated partitions" — debe ser 0 en estado normal.

Métrica para detectar broker no disponible: - `up{job="kafka-jmx"} == 0` — Prometheus marca el target como down si no responde el scrape. - O

`count(kafka_server_replicamanager_underreplicatedpartitions) by (instance)` que aumentará si un broker se cae.

Group con mayor lag: típicamente el del Sink (`connect-novatech-sink-procesados`) si lo detuviste. El de ksqldb (`_confluent-ksql-novatech_query_*`) suele tener lag bajo.

Desafío 4: diseño seguro del pipeline

Esto es subjetivo, pero las respuestas mínimas esperadas:

Lab 09 (Kafka Connect + PostgreSQL)

- Connect autenticado al broker via SASL_SSL con principal `connect-user`.
- ACLs:
- Read/Write/Describe sobre `connect-configs`, `connect-offsets`, `connect-status`.
- Read sobre topics origen, Write sobre topics destino.
- PostgreSQL: usuario `novatech-readonly` para el source, con SELECT solo en las tablas necesarias.
- Connection string a PostgreSQL con `ssl=true` y truststore cargado.

Lab 10 (Schema Registry + ksqldb)

- Schema Registry con HTTPS (cert generado con la misma CA que el broker).
- Schema Registry con autenticación basic (usuario `sr-reader` para los clientes, `sr-writer` para apps que registren schemas).
- ksqldb con su propio principal `ksqldb-user` con ACLs sobre topics que crea (`_confluent-ksql-*`) y topics de origen.
- Conexión ksqldb → Schema Registry vía HTTPS con auth.

Lab 11 (Prometheus + JMX exporter)

- JMX exporter HTTP detrás de auth básica (`prometheus.yml` con `basic_auth`).
- Prometheus tras reverse proxy con TLS terminator.
- Grafana con SSO en lugar de admin/admin. `GF_AUTH_ANONYMOUS_ENABLED=false`.

Cluster en general

- TLS en TODOS los listeners (incluido el CONTROLLER).

- SASL/SCRAM-SHA-512 en lugar de PLAIN.
- mTLS para inter-broker (el listener INTERNAL deja de ser PLAINTEXT).
- super.users acotado a un único admin humano más cuentas de servicio justificadas.
- Rotación automática de certificados (Vault PKI, cert-manager).
- Auditoría: `kafka-acls --list` exportado periódicamente a un repo o sistema de gestión.

Desafío 5 (opcional): conectar redes Docker

```
# Conectar el container ksqldb-server a la red del Lab 09
docker network connect novatech-lab09-net ksqldb-server

# Verificar
docker inspect ksqldb-server --format '{{json .NetworkSettings.Networks}}' | jq

# Ahora desde la CLI de ksqlDB puedes apuntar a los brokers del Lab 09:
# El bootstrap sería: kafka-broker-1:29092 (broker del Lab 09)
# PERO ojo: hay colisión de hostnames si Lab 10 también tiene "kafka-broker-1".
```

Problema realista que vas a chocar: ambos labs nombran a sus brokers `kafka-broker-1`, `kafka-broker-2`, `kafka-broker-3`. Si conectas ksqlDB a la red del Lab 09, el DNS resolverá al broker que Docker decida. Soluciones:

1. **Renombrar los containers** del Lab 09 a `kafka-09-broker-1`, etc. (cambia el compose).
2. **Usar IPs directamente:** `docker inspect kafka-broker-1` (en la red del Lab 09) y usar la IP. Frágil porque cambia al recrear.
3. **El approach realista en producción:** distintos clusters tienen distintos endpoints (DNS records, no nombres de container).

Conclusión pedagógica: por eso en producción los labs y entornos viven en DNS distinto y no se cruzan por nombre de container.